SERVA

Technical Architecture Document

Version 1.0 — April 2026
Confidential

1. System Architecture Overview

Serva is built on a modern, event-driven microservices architecture hosted on AWS. The system is designed for horizontal scalability, fault isolation, and rapid feature iteration. Each core domain (booking, payments, search, users, notifications) operates as an independent service with its own data store, communicating through both synchronous APIs and asynchronous event streams.

Architecture Principles

- **API-First:** Every feature is accessible through a well-documented REST/GraphQL API before any UI is built. This enables mobile apps, web apps, partner integrations, and third-party developers to access the same capabilities.
- **Event-Driven:** State changes emit domain events through Apache Kafka. Services react to events asynchronously, enabling loose coupling and eventual consistency where appropriate.
- **Mobile-First:** The consumer experience is designed for mobile screens first, with responsive web as a secondary channel. Native apps (React Native) provide the primary interface.
- **Multi-Tenant by Design:** Each service provider's data is logically isolated within shared infrastructure. Tenant context is propagated through every request for security and data integrity.
- **Security-First:** Zero-trust networking, encrypted data at rest and in transit, RBAC (role-based access control), and compliance with SOC 2 Type II, PCI DSS, and HIPAA where applicable.

2. Technology Stack

Layer	Technology	Rationale
Mobile Apps	React Native + Expo	Single codebase for iOS & Android. 90%+ code sharing. Hot updates via OTA.
Web App	Next.js (React) + TypeScript	SSR for SEO (business profiles), fast load times, shared component library with mobile.
API Gateway	AWS API Gateway + Kong	Rate limiting, auth, routing, request transformation, API versioning.
Backend Services	Node.js + TypeScript (NestJS)	Strong typing, dependency injection, modular architecture. Massive ecosystem.
Primary Database	PostgreSQL (AWS RDS)	ACID transactions for bookings/payments. JSON support for flexible schemas.
Cache Layer	Redis (AWS ElastiCache)	Session storage, rate limiting, real-time availability caching, pub/sub.
Search Engine	Elasticsearch (OpenSearch)	Full-text search, geo-spatial queries, faceted filtering, relevance tuning.
Event Streaming	Apache Kafka (AWS MSK)	Event sourcing, service communication, analytics pipeline, audit trail.
Object Storage	AWS S3 + CloudFront CDN	Business photos, documents, intake form uploads. Global CDN for fast delivery.
Payments	Stripe Connect	Split payments, marketplace payouts, PCI compliance, fraud detection built-in.
Push Notifications	Firebase Cloud Messaging + AWS SNS	Cross-platform push, email, SMS. Reliable delivery with fallback.
Authentication	AWS Cognito + OAuth 2.0	Social login (Google, Apple, Facebook), MFA, JWT-based session management.
CI/CD	GitHub Actions + AWS CodePipeline	Automated testing, staging deployments, blue-green production releases.
Infrastructure	Terraform + AWS ECS (Fargate)	Infrastructure as code. Serverless containers. No server management.
Monitoring	Datadog + PagerDuty	APM, log aggregation, real-time alerting, incident management.

3. Core Microservices

The Serva platform is decomposed into the following core services, each owning its domain logic and data:

3.1 User Service

Manages all user identities, authentication, and profile data for both consumers and service providers. Handles registration, login, social auth, MFA, and profile management.

- **Database:** PostgreSQL (users, profiles, preferences, saved payment methods reference)
- **Key APIs:** POST /auth/register, POST /auth/login, GET /users/:id, PUT /users/:id/profile
- **Events Emitted:** user.created, user.updated, user.verified

3.2 Business Service

Manages service provider profiles, service menus, pricing, photos, and business settings. This is the “digital storefront” engine.

- **Database:** PostgreSQL (businesses, services, pricing, staff, locations, operating hours)
- **Key APIs:** POST /businesses, GET /businesses/:id, PUT /businesses/:id/services, GET /businesses/:id/menu
- **Events Emitted:** business.created, business.updated, service.added, service.updated

3.3 Booking Service

The core scheduling engine. Manages availability slots, appointment creation, modifications, cancellations, and no-show tracking. Handles complex scheduling rules including staff assignment, resource allocation, buffer times, and recurring appointments.

- **Database:** PostgreSQL (bookings, availability_slots, scheduling_rules, waitlists)
- **Key APIs:** GET /availability/:businessId, POST /bookings, PUT /bookings/:id, DELETE /bookings/:id
- **Events Emitted:** booking.created, booking.confirmed, booking.cancelled, booking.completed, booking.noshow
- **Real-Time:** WebSocket connections for live availability updates. Redis pub/sub for slot locking during checkout.

3.4 Payment Service

Handles all financial transactions through Stripe Connect. Manages payment intents, split payments (platform fee + provider payout), refunds, deposits, tips, and cancellation fees.

- **Database:** PostgreSQL (transactions, payment_methods, payouts, refunds, invoices)
- **Key APIs:** POST /payments/intent, POST /payments/capture, POST /refunds, GET /payouts/:businessId
- **Events Emitted:** payment.completed, payment.failed, refund.issued, payout.sent
- **PCI Compliance:** No raw card data touches Serva servers. Stripe.js and Stripe Mobile SDK handle tokenization client-side.

3.5 Search & Discovery Service

Powers the consumer search experience with geo-spatial search, full-text queries, category filtering, price range filtering, availability-based filtering, and AI-powered recommendations.

- **Database:** Elasticsearch (business index, service index, geo index)
- **Key APIs:** GET /search?q=&lat=&lng=&category=&priceRange=&available=
- **Features:** Typo tolerance, synonym matching, boost factors (rating, proximity, availability), personalized ranking based on user history.

3.6 Review & Rating Service

Manages verified reviews (only users who completed a booking can review), star ratings, review responses from businesses, and aggregate rating calculations.

- **Database:** PostgreSQL (reviews, ratings, responses, helpfulness_votes)
- **Key APIs:** POST /reviews, GET /businesses/:id/reviews, POST /reviews/:id/response
- **Events Emitted:** review.created, review.flagged, rating.updated

3.7 Notification Service

Centralized notification engine handling push notifications, email, SMS, and in-app messages. Manages notification preferences, templates, and delivery tracking.

- **Channels:** Push (FCM/APNs), Email (AWS SES), SMS (Twilio), In-App
- **Key Events Consumed:** booking.created → confirmation to both parties, booking.reminder → 24hr/1hr before, payment.completed → receipt

3.8 Intake Form Service

Manages the customizable client information forms that businesses create and consumers fill out before appointments. Supports various field types (text, dropdown, date, file upload, signature, checkbox) and conditional logic.

- **Database:** PostgreSQL + S3 (form_templates, form_submissions, uploaded_files)
- **Key APIs:** POST /forms/templates, GET /forms/templates/:businessId, POST /forms/submissions
- **HIPAA Note:** For healthcare verticals, form submissions are encrypted at rest with customer-managed keys (AWS KMS) and access is audit-logged.

4. Database Schema (Core Entities)

Below are the primary database entities and their relationships. Each microservice owns its schema, with cross-service references using UUIDs.

4.1 Users Table

Column	Type	Constraints	Description
id	UUID	PK	Unique user identifier
email	VARCHAR(255)	UNIQUE, NOT NULL	Login email
phone	VARCHAR(20)	UNIQUE	Phone number (E.164)
first_name	VARCHAR(100)	NOT NULL	First name
last_name	VARCHAR(100)	NOT NULL	Last name
avatar_url	TEXT		Profile photo URL
role	ENUM	NOT NULL	consumer, provider, admin
stripe_customer_id	VARCHAR(255)		Stripe customer reference
created_at	TIMESTAMPTZ	NOT NULL	Account creation date

4.2 Businesses Table

Column	Type	Constraints	Description
id	UUID	PK	Business identifier
owner_id	UUID	FK → users.id	Business owner
name	VARCHAR(255)	NOT NULL	Business name
slug	VARCHAR(255)	UNIQUE	URL-friendly identifier
category_id	UUID	FK → categories.id	Primary category
description	TEXT		Business description
address	JSONB	NOT NULL	Structured address object
location	GEOGRAPHY(POINT)	NOT NULL	Geo coordinates (PostGIS)
phone	VARCHAR(20)		Business phone
operating_hours	JSONB		Weekly schedule
subscription_tier	ENUM	DEFAULT free	free, pro, business, enterprise

stripe_account_id	VARCHAR(255)		Stripe Connect account
avg_rating	DECIMAL(3,2)	DEFAULT 0	Denormalized avg rating
total_reviews	INTEGER	DEFAULT 0	Denormalized review count

4.3 Services Table

Column	Type	Constraints	Description
id	UUID	PK	Service identifier
business_id	UUID	FK → businesses.id	Parent business
name	VARCHAR(255)	NOT NULL	Service name
description	TEXT		Service description
price_cents	INTEGER	NOT NULL	Price in cents
price_type	ENUM	NOT NULL	fixed, starting_at, hourly, free
duration_minutes	INTEGER	NOT NULL	Appointment duration
buffer_minutes	INTEGER	DEFAULT 0	Buffer between appointments
deposit_required	BOOLEAN	DEFAULT false	Requires deposit to book
deposit_amount_cents	INTEGER		Deposit amount if required
intake_form_id	UUID	FK → form_templates.id	Required intake form
is_active	BOOLEAN	DEFAULT true	Listed/unlisted

4.4 Bookings Table

Column	Type	Constraints	Description
id	UUID	PK	Booking identifier
consumer_id	UUID	FK → users.id	Customer who booked
business_id	UUID	FK → businesses.id	Service provider
service_id	UUID	FK → services.id	Selected service
staff_id	UUID	FK → staff.id	Assigned staff member
start_time	TIMESTAMPTZ	NOT NULL	Appointment start

end_time	TIMESTAMPTZ	NOT NULL	Appointment end
status	ENUM	NOT NULL	pending, confirmed, in_progress, completed, cancelled, no_show
total_cents	INTEGER	NOT NULL	Total price
payment_status	ENUM	NOT NULL	pending, authorized, captured, refunded
intake_submission_id	UUID	FK → form_submissions.id	Completed intake form
notes	TEXT		Special requests
cancellation_reason	TEXT		If cancelled

5. API Design

All APIs follow RESTful conventions with consistent response formats, pagination, error handling, and versioning through URL prefixes (/api/v1/).

5.1 Authentication Flow

All API requests require a valid JWT bearer token. Tokens are issued by the User Service upon successful authentication and expire after 1 hour, with refresh tokens valid for 30 days.

```
Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...
```

5.2 Core API Endpoints

Method	Endpoint	Description
POST	/api/v1/auth/register	Register new user (consumer or provider)
POST	/api/v1/auth/login	Authenticate and receive JWT
GET	/api/v1/search	Search businesses by query, location, category, filters
GET	/api/v1/businesses/:id	Full business profile with services, reviews, photos
GET	/api/v1/businesses/:id/services	Service menu with pricing and availability
GET	/api/v1/availability/:businessId	Available time slots for a specific service and date range
POST	/api/v1/bookings	Create a new booking (reserves slot, initiates payment)
PUT	/api/v1/bookings/:id	Modify booking (reschedule, update notes)
DELETE	/api/v1/bookings/:id	Cancel booking (triggers refund policy)
POST	/api/v1/payments/intent	Create payment intent for booking
POST	/api/v1/forms/:id/submit	Submit completed intake form
POST	/api/v1/reviews	Submit review for completed booking

5.3 Standard Response Format

```
{ "status": "success", "data": { ... }, "meta": { "page": 1, "per_page": 20, "total": 150 } }
```

5.4 Error Response Format

```
{ "status": "error", "error": { "code": "SLOT_UNAVAILABLE", "message": "The selected time slot is no longer available.", "details": {} } }
```

6. Payment Architecture

Payments are the lifeblood of Serva. The payment system is built on Stripe Connect in platform (destination charges) mode, which handles the complexity of marketplace payments while maintaining PCI compliance.

6.1 Payment Flow

- **Step 1 — Consumer Selects Service:** Price is displayed from the service menu. Consumer proceeds to booking.
- **Step 2 — Payment Intent Created:** Serva creates a Stripe PaymentIntent with the total amount, application fee (2.5%), and the provider's connected Stripe account as destination.
- **Step 3 — Card Tokenized Client-Side:** Stripe.js or Stripe Mobile SDK collects card details and returns a token. No raw card data reaches Serva servers.
- **Step 4 — Payment Authorized:** For bookings with future dates, the payment is authorized (held) but not captured until the appointment is completed or a configurable capture window.
- **Step 5 — Payment Captured:** Upon appointment completion, the payment is captured. The platform fee is retained by Serva, and the remainder is queued for payout to the provider.
- **Step 6 — Provider Payout:** Stripe sends funds to the provider's bank account on a configurable schedule (daily, weekly, or monthly).

6.2 Payment Features

- **Saved Payment Methods:** Consumers can save multiple cards and select their preferred method at checkout. Powered by Stripe Customer objects.
- **Deposits:** Providers can require a deposit (fixed amount or percentage) at booking time, with the balance due at the appointment.
- **Cancellation Fees:** Configurable cancellation policies (free cancellation window, late cancellation fees, no-show charges).
- **Tips:** Post-appointment tipping through the app, with 100% of tips going to the provider.
- **Refunds:** Full and partial refunds processed through Stripe, with automatic reconciliation.

7. Infrastructure & DevOps

7.1 Cloud Architecture (AWS)

- **Compute:** ECS Fargate (serverless containers) for all microservices. Auto-scaling based on CPU/memory utilization and request count.
- **Database:** RDS PostgreSQL with Multi-AZ deployment, automated backups, and read replicas for scaling read-heavy workloads.
- **Cache:** ElastiCache Redis cluster with automatic failover.
- **Search:** OpenSearch (managed Elasticsearch) with dedicated master nodes and data nodes.
- **Messaging:** MSK (managed Kafka) for event streaming with 3-broker minimum.
- **CDN:** CloudFront for static assets, business photos, and API edge caching.
- **DNS:** Route 53 with health checks and latency-based routing.

7.2 Environments

Environment	Purpose	Infrastructure
Development	Individual developer workstations	Docker Compose, local PostgreSQL, local Redis
Staging	Pre-production testing, QA	Full AWS stack at reduced scale
Production	Live customer traffic	Full AWS stack, multi-AZ, auto-scaling

7.3 Deployment Strategy

Blue-green deployments for zero-downtime releases. Each service can be deployed independently. Feature flags (LaunchDarkly) enable gradual rollouts and instant rollbacks without redeployment.

7.4 Monitoring & Observability

- **Metrics:** Datadog APM for request latency, error rates, throughput per service. Custom business metrics (bookings/minute, payment success rate).
- **Logs:** Structured JSON logging, centralized in Datadog Logs with retention and alerting.
- **Traces:** Distributed tracing (OpenTelemetry) across all services for end-to-end request debugging.

- **Alerts:** PagerDuty integration with escalation policies. P1 (site down): 5-min response. P2 (degraded): 30-min response.

8. Security & Compliance

8.1 Data Security

- **Encryption at Rest:** AES-256 for all databases, S3 objects, and backups. Customer-managed KMS keys for healthcare data.
- **Encryption in Transit:** TLS 1.3 for all API communication. Certificate pinning in mobile apps.
- **Secrets Management:** AWS Secrets Manager for API keys, database credentials, and third-party tokens. No secrets in code or environment variables.

8.2 Access Control

- **Authentication:** OAuth 2.0 + OIDC. JWT tokens with RS256 signing. Refresh token rotation.
- **Authorization:** RBAC with four roles: consumer, provider, provider_staff, admin. Fine-grained permissions per API endpoint.
- **API Security:** Rate limiting, input validation, OWASP Top 10 protections, SQL injection prevention via parameterized queries.

8.3 Compliance

- **PCI DSS:** Level 1 compliance through Stripe. No raw card data stored or processed by Serva.
- **SOC 2 Type II:** Target certification within 12 months of launch. Covers security, availability, and confidentiality.
- **HIPAA:** BAA with Stripe and AWS. Encrypted PHI storage, audit logging, and access controls for healthcare verticals.
- **GDPR/CCPA:** User data export, right to deletion, consent management, and data processing agreements.

9. Scalability Roadmap

The architecture is designed to scale from hundreds to millions of bookings per day without re-architecture.

Scale	Bookings/Day	Infrastructure	Key Changes
Launch	500–2,000	2 ECS tasks/service, single RDS	Baseline configuration
Growth	2,000–50,000	Auto-scaling, RDS read replicas	Add caching layers, CDN optimization
Scale	50,000–500,000	Multi-region, sharded databases	Database sharding by geography, dedicated search clusters
Hyper-Scale	500,000+	Global CDN, edge computing	Edge availability checks, regional data residency

End of Technical Architecture Document